Manual técnico de RFMSU VI / RFMSU VI Technical Manual

Arquitectura, validación y ejemplos / Architecture, validation and examples

Miguel García Torres

23 de octubre de 2026

Resumen

Este manual documenta Random Forest MSU, la cadena de dependencias, las clases y métodos, las baterías de validación, los datos suministrados y seis ejemplos completos y ejecutables.

Índice general

1. Manual en español **3**
- 1.1. Ficheros . . . 3
- 1.2. Arquitectura, herencia y dependencias . . . 3
- 1.3. Inventario de clases y métodos . . . 4
  - 1.3.1. `RandomForestMSU` — `random_forest_msu.py` . . . 4
  - 1.3.2. `Tree` — `random_forest.py` . . . 4
  - 1.3.3. `TreeClassifier` — `random_forest.py` . . . 4
  - 1.3.4. `RandomForest` — `random_forest.py` . . . 4
  - 1.3.5. `TreeMSU` — `random_tree_msu.py` . . . 4
  - 1.3.6. `TreeClassifierMSU` — `random_tree_msu.py` . . . 4
  - 1.3.7. `Tree` — `random_tree.py` . . . 4
  - 1.3.8. `TreeClassifier` — `random_tree.py` . . . 4
  - 1.3.9. `MDLBasedDiscretizationModel` — `disc_utils.py` . . . 4
  - 1.3.10. `MDLBasedDiscretization` — `disc_utils.py` . . . 5
  - 1.3.11. `FayyadIranisDiscretization` — `disc_utils.py` . . . 5
- 1.4. Los datos utilizados . . . 5
- 1.5. Pruebas unitarias, matemáticas y de referencia . . . 5
  - 1.5.1. Pruebas unitarias — `random_forest_msu_test.py` . . . 5
  - 1.5.2. Matemáticas — `random_forest_msu_test_math.py` . . . 5
  - 1.5.3. Científicas — `random_forest_msu_test_ref.py` . . . 5
  - 1.5.4. Ejecución reproducida y límites . . . 5
- 1.6. Resultados de los ejemplos sobre lymphoma . . . 6
- 1.7. Ejemplos Python con resaltado de sintaxis . . . 6
  - 1.7.1. comun.py . . . 6
  - 1.7.2. 01_csv_train_test.py . . . 7
  - 1.7.3. 02_csv_validacion_cruzada.py . . . 7
  - 1.7.4. 03_configuraciones.py . . . 8
  - 1.7.5. 04_arff_train_test.py . . . 8
  - 1.7.6. 05_arff_validacion_cruzada.py . . . 8
  - 1.7.7. 06_csv_todos_atributos.py . . . 9
- 1.8. Notas de validez, recomendaciones y limitaciones . . . 9

2. Technical manual in English **10**
- 2.1. Scope, files, and reproducibility . . . 10
- 2.2. Architecture, inheritance, and dependencies . . . 10
- 2.3. Classes and principal methods . . . 11
  - 2.3.1. `RandomForestMSU` — `random_forest_msu.py` . . . 11
  - 2.3.2. `Tree` — `random_forest.py` . . . 11
  - 2.3.3. `TreeClassifier` — `random_forest.py` . . . 11
  - 2.3.4. `RandomForest` — `random_forest.py` . . . 11
  - 2.3.5. `TreeMSU` — `random_tree_msu.py` . . . 11
  - 2.3.6. `TreeClassifierMSU` — `random_tree_msu.py` . . . 12

2.3.7. `Tree` — `random_tree.py` . . . 12
2.3.8. `TreeClassifier` — `random_tree.py` . . . 12
2.3.9. `MDLBasedDiscretizationModel` — `disc_utils.py` . . . 12
2.3.10. `MDLBasedDiscretization` — `disc_utils.py` . . . 12
2.3.11. `FayyadIranisDiscretization` — `disc_utils.py` . . . 12
2.4. Datasets . . . 12
2.5. Unit tests, mathematical verification, and scientific references . . . 12
2.5.1. Unit tests — `random_forest_msu_test.py` . . . 12
2.5.2. Mathematical tests — `random_forest_msu_test_math.py` . . . 13
2.5.3. Scientific-library comparisons — `random_forest_msu_test_ref.py` . . . 13
2.6. Lymphoma example results . . . 13
2.7. Runnable Python examples with syntax highlighting . . . 13
2.7.1. `comun.py`: shared helper functions . . . 14
2.7.2. `01_csv_train_test.py`: separate training and test CSV files . . . 14
2.7.3. `02_csv_validacion_cruzada.py`: CSV cross-validation . . . 15
2.7.4. `03_configuraciones.py`: different MSU and bootstrap configurations . . . 15
2.7.5. `04_arff_train_test.py`: separate training and test ARFF files . . . 16
2.7.6. `05_arff_validacion_cruzada.py`: ARFF cross-validation . . . 16
2.7.7. `06_csv_todos_atributos.py`: all 2,647 predictor columns . . . 16
2.8. Validity notes, recommendations, and limitations . . . 17

Capítulo 1

Manual en español

1.1. Ficheros

La carpeta `python/` contiene la implementación.

```
python -m pip install numpy pandas scipy scikit-learn pytest pytest-cov joblib
cd manual_rfmsu
PYTHONPATH=codigo:ejemplos python ejemplos/01_csv_train_test.py
cd codigo
python -m pytest -v random_forest_msu_test.py \
 random_forest_msu_test_math.py random_forest_msu_test_ref.py \
 --cov=random_forest_msu --cov-branch --cov-report=term-missing
```

1.2. Arquitectura, herencia y dependencias

La clase `RandomForestMSU` extiende `RandomForest` y sobrescribe la construcción y ajuste de los clasificadores base, que son instancias de `TreeClassifierMSU`. Esta clase extiende `TreeClassifier` de `random_tree.py`; `TreeMSU` extiende `Tree` del mismo archivo. `RandomForest` incluye también sus propias clases `Tree` y `TreeClassifier` (no VI) para su modo genérico, lo que constituye una *duplicación arquitectónica*, pero la instancia VI de RFMSU emplea la rama corregida.

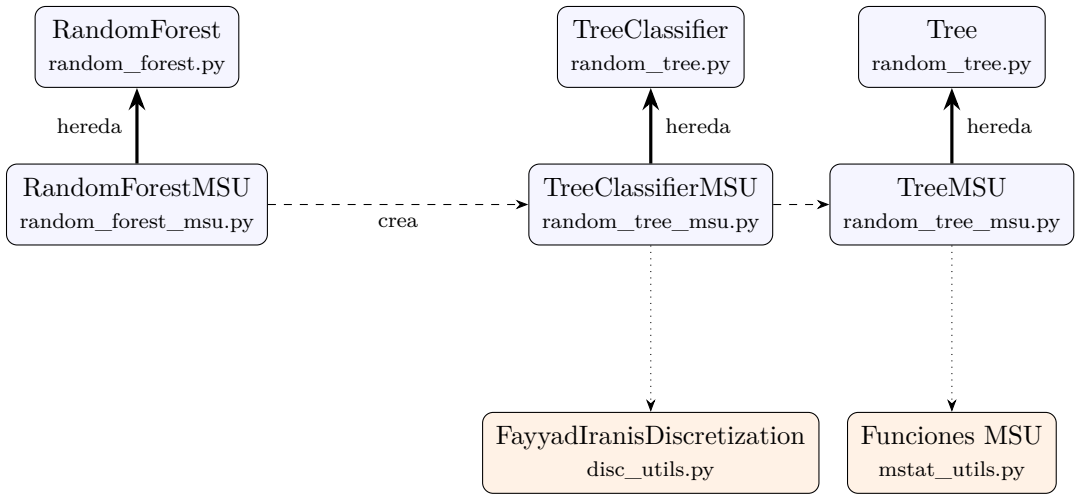


Figura 1.1: Herencia (líneas continuas) y relaciones de uso (discontinuas/punteadas). La flecha de herencia apunta a la clase base. Diagrama esquemático.

Las utilidades `stat_utils.py` aportan entropías; `mstat_utils.py` aporta MSU; `disc_utils.py` aporta discretización MDL/Fayyad–Irani. La paralelización utiliza `joblib` cuando está disponible;

numba es opcional para determinadas operaciones del discretizador.

1.3. Inventario de clases y métodos

1.3.1. RandomForestMSU — random_forest_msu.py

Clase base: RandomForest. Métodos definidos directamente en esta clase: __post_init__, _make_tree, _fit_msu_tree_from_bootstrap_spec, fit, get_params, set_params, to_string, __str__.

1.3.2. Tree — random_forest.py

Clase base: ninguna explícita. Métodos definidos directamente en esta clase: weka_gr, weka_eq, is_missing_value, gain, prior_val, distribution, split_data, _normalized_distribution, distribution_fo build_tree, _leaf_string, _format_number, to_string, num_nodes, num_leaves, used_attributes, summary_string.

1.3.3. TreeClassifier — random_forest.py

Clase base: ninguna explícita. Métodos definidos directamente en esta clase: single_variance, is_missing_value, infer_feature_types, get_categorical_values, fit, to_string, num_nodes, num_leaves, used_attributes, used_attribute_names, distribution_for_instance, predict_proba, predict.

1.3.4. RandomForest — random_forest.py

Clase base: ninguna explícita. Métodos definidos directamente en esta clase: __post_init__, _rng_seed, _validate_X_y, _make_tree, _fit_tree_from_bootstrap_spec, _aligned_tree_proba, fit, _compute_oob_score predict_proba, predict, distribution_for_instance, get_estimators, getClassifiers, compute_average_imp computeAverageImpurityDecreasePerAttribute, compute_average_metrics, computeAverageMetrics, get_params, set_params, to_string, __str__.

1.3.5. TreeMSU — random_tree_msu.py

Clase base: Tree. Métodos definidos directamente en esta clase: weka_gr, weka_eq, _discretize_msu_for_node, distribution, _extend_msu_subset, _msu_score, split_data_msu, build_tree_msu.

1.3.6. TreeClassifierMSU — random_tree_msu.py

Clase base: TreeClassifier. Métodos definidos directamente en esta clase: _is_missing_value, _discretize_with_u prepare_msu_discrete, fit.

1.3.7. Tree — random_tree.py

Clase base: ninguna explícita. Métodos definidos directamente en esta clase: weka_gr, weka_eq, is_missing_value, gain, prior_val, distribution, split_data, _normalized_distribution, distribution_fo build_tree, _leaf_string, _format_number, to_string, num_nodes, num_leaves, used_attributes, summary_string.

1.3.8. TreeClassifier — random_tree.py

Clase base: ninguna explícita. Métodos definidos directamente en esta clase: single_variance, is_missing_value, infer_feature_types, get_categorical_values, fit, to_string, num_nodes, num_leaves, used_attributes, used_attribute_names, distribution_for_instance, predict_proba, predict.

1.3.9. MDLBasedDiscretizationModel — disc_utils.py

Clase base: ninguna explícita. Métodos definidos directamente en esta clase: __init__, get_cut_points, set_cut_points, __str__, discretize, transform.

1.3.10. `MDLBasedDiscretization` — `disc_utils.py`

Clase base: `ABC`. Métodos definidos directamente en esta clase: `__init__`, `build_discretization_model`, `_calculate_feature_cut_points`, `calculate_cut_points`, `split_test`.

1.3.11. `FayyadIranisDiscretization` — `disc_utils.py`

Clase base: `MDLBasedDiscretization`. Métodos definidos directamente en esta clase: `split_test`.

1.4. Los datos utilizados

Se han inspeccionado los tres CSV adjuntos: el conjunto completo contiene 77 observaciones (58 de clase 1 y 19 de clase 0); el de entrenamiento, 39 (29 y 10), y el de test, 38 (29 y 9). Cada archivo tiene 2.648 columnas: 2.647 características numéricas y la columna `class`. Se mantiene la partición train/test original sin reentrenar con observaciones del test. Los ARFF del paquete son demostraciones derivadas de estos CSV que contienen las primeras 20 características y la clase; *no representan archivos ARFF aportados por el usuario.*

Conjunto	Filas	Atributos	Clases (0/1)
Completo	77	2647	19/58
Entrenamiento	39	2647	10/29
Test	38	2647	9/29

1.5. Pruebas unitarias, matemáticas y de referencia

1.5.1. Pruebas unitarias — `random_forest_msu_test.py`

Verifican herencia, tipos de estimadores, semillas y reproducibilidad, formas de salida, modalidades `sample`/`weighted`, conteos e índices bootstrap, caché de discretización global, matrices discretizadas explícitas, validación de parámetros, OOB y equivalencia secuencial/paralela, así como regresiones de categorías desconocidas y ramas vacías en VI.

1.5.2. Matemáticas — `random_forest_msu_test_math.py`

La selección de índices bootstrap debe cumplir $\sum_i c_i = B$, donde B es el tamaño de bolsa. La agregación probabilística normal es $\widehat{P}(c \mid x) = T^{-1} \sum_{t=1}^{T} \widehat{P}_t(c \mid x)$, alineando clases locales y globales. El modo votación usa $T^{-1} \sum_t \mathbf{1}\{h_t(x) = c\}$. La predicción OOB de cada observación se calcula exclusivamente con los árboles cuya bolsa no la contiene. Estas identidades se comprueban con reconstrucciones independientes.

1.5.3. Científicas — `random_forest_msu_test_ref.py`

Se contrastan las formas de predicción, etiquetas y rendimiento de un problema controlado frente a `RandomForestClassifier` de scikit-learn. El modelo de referencia utiliza CART y no debe producir necesariamente los mismos splits que MSU; esta comparación es de comportamiento y desempeño, *no* de identidad algorítmica. SciPy permite contrastar la probabilidad teórica de que una instancia quede OOB: $\Pr(C_i = 0) = (1 - 1/N)^B$.

1.5.4. Ejecución reproducida y límites

La ejecución conjunta de las tres baterías para `random_forest_msu.py` obtuvo **27 passed**, **0 failed** y **88 % de cobertura combinada (instrucciones y ramas)**: 146 instrucciones, 16 no ejecutadas; 48 ramas, 8 parcialmente cubiertas. Prueba efectuada en Python 3.13.5. La cobertura mide solo ese módulo y no incluye automáticamente todo el código de sus dependencias. Las pruebas anteriores de otros módulos no se han vuelto a ejecutar como parte de esta medición.

```
python -m pytest -v random_forest_msu_test.py \
 random_forest_msu_test_math.py random_forest_msu_test_ref.py \
 --cov=random_forest_msu --cov-branch \
 --cov-report=term-missing --cov-report=html
```

1.6. Resultados de los ejemplos sobre lymphoma

Todas las medidas mostradas se obtuvieron ejecutando los scripts incluidos y usando 20 predictores, cinco árboles y las semillas definidas en cada archivo. La partición original tiene una clase mayoritaria notable: conviene interpretar tanto accuracy como balanced accuracy.

Ejecución	Accuracy	Balanced accuracy
CSV train/test, global+sample	0.8158	0.6111
CSV train/test, local+sample	0.6842	0.4483
CSV train/test, global+weighted	0.7632	0.5383
ARFF train/test, global+sample	0.8158	0.6111
CV estratificada 5 folds (media)	0.8200	0.7150

El OOB score ilustrativo para la configuración global+sample fue 0.8333; para local+sample, 0.5278; y para global+weighted, 0.6944. Los resultados varían con la selección de atributos y semillas; no constituyen un benchmark concluyente.

1.7. Ejemplos Python con resaltado de sintaxis

Las palabras clave aparecen en azul, cadenas en naranja, comentarios en verde y nombres de clases relevantes en violeta. Los listings siguientes muestran el *contenido literal* de los scripts entregados; se pueden ejecutar individualmente. En todos los casos, el atributo objetivo se denomina `class`.

1.7.1. comun.py

Funciones auxiliares: carga CSV/ARFF, selección de variables, configuración del clasificador y métricas.

```
"""Lectura reproducible de datos y construccion de RandomForestMSU VI.

Por defecto usa 20 atributos predictivos para demostraciones rapidas.
Para usar todos los atributos: --max-features 0.
"""
from pathlib import Path
import numpy as np
import pandas as pd
from scipy.io import arff
from rfmsu import RandomForestMSU

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / 'datos'

def select_features(frame, max_features=20):
    if 'class' not in frame.columns:
        raise ValueError('No se encontro la columna objetivo class')
    names = [c for c in frame if c != 'class']
    if max_features > 0:
        names = names[:max_features]
    return names

def load_csv(name, features=None, max_features=20):
    df = pd.read_csv(DATA / name)
    if features is None:
        features = select_features(df, max_features)
    X = df.loc[:, features].to_numpy(dtype=float)
```

```
    y = df['class'].to_numpy()
    return X, y, features

def load_arff(name, features=None, max_features=20):
    records, _ = arff.loadarff(DATA / name)
    df = pd.DataFrame(records)
    for col in df.columns:
        if df[col].dtype == object:
            df[col] = df[col].map(lambda v: v.decode('utf8') if isinstance(v, bytes) else v)
    if features is None:
        features = select_features(df, max_features)
    X = df.loc[:, features].to_numpy(dtype=float)
    y = df['class'].to_numpy()
    return X, y, features

def model(*, trees=5, seed=23, bootstrap='sample', local=False, oob=False,
          n_jobs=1, majority=False):
    return RandomForestMSU(
        debug=False, numIterations=trees, seed=seed, kValue=3,
        bootstrap_mode=bootstrap, rediscretize_msu_per_node=local,
        calcOutOfBag=oob, n_jobs=n_jobs, majority_vote=majority,
    )

def report(y_true, y_pred):
    from sklearn.metrics import accuracy_score, balanced_accuracy_score, confusion_matrix
    print('Accuracy:', round(accuracy_score(y_true, y_pred), 4))
    print('Balanced accuracy:', round(balanced_accuracy_score(y_true, y_pred), 4))
    print('Confusion matrix:\n', confusion_matrix(y_true, y_pred))
```

Listing 1.1: comun.py

1.7.2. 01_csv_train_test.py

Entrena con el CSV train y evalúa exclusivamente en el CSV test; también comprueba el número de probabilidades y de árboles.

```
"""Entrenamiento CSV y evaluacion en un CSV de test independiente."""
import argparse
from comun import load_csv, model, report

parser = argparse.ArgumentParser()
parser.add_argument('--max-features', type=int, default=20)
parser.add_argument('--trees', type=int, default=5)
args = parser.parse_args()

X_train, y_train, features = load_csv(
    '04-lymphoma_cancer_shipp_train.csv', max_features=args.max_features)
X_test, y_test, _ = load_csv(
    '04-lymphoma_cancer_shipp_test.csv', features=features)
clf = model(trees=args.trees, bootstrap='sample', local=False)
clf.fit(X_train, y_train, feature_types=['numeric'] * X_train.shape[1],
        feature_names=features)
pred = clf.predict(X_test)
report(y_test, pred)
print('Probability shape:', clf.predict_proba(X_test).shape)
print('Trees:', len(clf.estimators_))
```

Listing 1.2: 01_csv_train_test.py

1.7.3. 02_csv_validacion_cruzada.py

Realiza validación cruzada estratificada de 5 folds sobre el CSV completo. Cada `fit` aprende su discretización dentro de la partición de entrenamiento; evita fuga de datos.

```
"""Validacion cruzada estratificada sin discretizar fuera de cada fold."""
import argparse
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, balanced_accuracy_score
from comun import load_csv, model
```

```

parser = argparse.ArgumentParser()
parser.add_argument('--max-features', type=int, default=20)
parser.add_argument('--trees', type=int, default=5)
parser.add_argument('--folds', type=int, default=5)
args = parser.parse_args()
X, y, features = load_csv('04-lymphoma_cancer_shipp.csv',
                          max_features=args.max_features)
cv = StratifiedKFold(n_splits=args.folds, shuffle=True, random_state=2026)
acc, bacc = [], []
for fold, (train_idx, test_idx) in enumerate(cv.split(X, y), start=1):
    clf = model(trees=args.trees, seed=fold, local=False)
    clf.fit(X[train_idx], y[train_idx],
            feature_types=['numeric'] * len(features), feature_names=features)
    pred = clf.predict(X[test_idx])
    acc.append(accuracy_score(y[test_idx], pred))
    bacc.append(balanced_accuracy_score(y[test_idx], pred))
    print(f'Fold {fold}: accuracy={acc[-1]:.3f}, balanced={bacc[-1]:.3f}')
print(f'Accuracy media={np.mean(acc):.3f} +/- {np.std(acc):.3f}')
print(f'Balanced accuracy media={np.mean(bacc):.3f}')
```

Listing 1.3: 02_csv_validacion_cruzada.py

1.7.4. 03_configuraciones.py

Compara discretización local/global y bootstrap físico/ponderado, incluyendo OOB. El criterio MSU ponderado conserva su limitación conocida.

```
"""Comparar discretizacion local/global, bootstrap sample/weighted y OOB."""
from comun import load_csv, model, report

X_train, y_train, names = load_csv('04-lymphoma_cancer_shipp_train.csv')
X_test, y_test, _ = load_csv('04-lymphoma_cancer_shipp_test.csv',
                             features=names)
for bootstrap, local in [('sample', False), ('sample', True),
                         ('weighted', False)]:
    clf = model(trees=5, seed=23, bootstrap=bootstrap, local=local,
                oob=True, n_jobs=1)
    clf.fit(X_train, y_train, feature_types=['numeric'] * len(names))
    print('\nBootstrap:', bootstrap, '| local discretization:', local)
    report(y_test, clf.predict(X_test))
    print('OOB score:', clf.oob_score_)
# Nota: en modo weighted, el MSU se calcula con frecuencias no ponderadas.
```

Listing 1.4: 03_configuraciones.py

1.7.5. 04_arff_train_test.py

Carga dos archivos ARFF derivados del train/test mediante SciPy; entrena y evalúa con las mismas variables.

```
"""Leer ARFF train/test con scipy.io.arff y evaluar el bosque."""
from comun import load_arff, model, report

X_train, y_train, features = load_arff('lymphoma_train_demo.arff')
X_test, y_test, _ = load_arff('lymphoma_test_demo.arff', features=features)
clf = model(trees=5, local=False)
clf.fit(X_train, y_train, feature_types=['numeric'] * len(features),
        feature_names=features)
report(y_test, clf.predict(X_test))
```

Listing 1.5: 04_arff_train_test.py

1.7.6. 05_arff_validacion_cruzada.py

Carga ARFF completo y aplica validación cruzada estratificada con reentrenamiento en cada fold.

```python
"""Leer ARFF completo y ejecutar validacion cruzada estratificada."""
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score
from comun import load_arff, model

X, y, features = load_arff('lymphoma_full_demo.arff')
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=2026)
scores = []
for fold, (tr, te) in enumerate(cv.split(X, y), start=1):
    clf = model(trees=5, seed=fold, bootstrap='sample', local=False)
    clf.fit(X[tr], y[tr], feature_types=['numeric'] * len(features))
    score = accuracy_score(y[te], clf.predict(X[te]))
    scores.append(score)
    print(f'Fold {fold}: {score:.3f}')
print('Mean CV accuracy:', float(np.mean(scores)))
```

Listing 1.6: 05_arff_validacion_cruzada.py

1.7.7. 06_csv_todos_atributos.py

Ejemplo con los 2647 atributos originales y un mayor número de árboles. El tiempo de ejecución puede ser muy elevado.

```python
"""Ejemplo de entrenamiento con todos los atributos (mas costoso)."""
from comun import load_csv, model, report
Xtr, ytr, names = load_csv('04-lymphoma_cancer_shipp_train.csv',
                           max_features=0)
Xte, yte, _ = load_csv('04-lymphoma_cancer_shipp_test.csv',
                       features=names)
clf = model(trees=10, local=False, n_jobs=1)
clf.fit(Xtr, ytr, feature_types=['numeric'] * len(names))
print('Number of predictors:', len(names))
report(yte, clf.predict(Xte))
```

Listing 1.7: 06_csv_todos_atributos.py

1.8. Notas de validez, recomendaciones y limitaciones

1. En `bootstrap_mode=weighted`, la MSU original se calcula sobre frecuencias de filas y no aplica directamente los pesos bootstrap; por eso se recomienda `sample` para los estudios en los que la selección debe respetar el remuestreo.

2. No debe aplicarse la discretización supervisada sobre todo el conjunto antes de dividir en folds: supondría fuga de información. Los scripts delegan la discretización a cada `fit`.

3. Es útil extender el benchmark con distintos números de árboles, selección de atributos, semillas, datasets y validación frente a Java/Weka. Comparar solo accuracy no demuestra equivalencia entre implementaciones.

4. Los ficheros ARFF incluidos son subconjuntos representativos para ilustrar el formato y no sustituyen los CSV originales.

Chapter 2

Technical manual in English

2.1. Scope, files, and reproducibility

This chapter is the English version of the preceding Spanish manual. It documents the Random Forest MSU implementation, its class relationships, validation suites, lymphoma datasets, and six runnable examples. The material is organized in the same order as the Spanish chapter; both chapters embed the *same Python files*, thereby preventing divergence between examples in the two languages.

The `python/` directory holds the implementation.

```
python -m pip install numpy pandas scipy scikit-learn pytest pytest-cov joblib
cd manual_rfmsu
PYTHONPATH=codigo:ejemplos python ejemplos/01_csv_train_test.py
cd codigo
python -m pytest -v random_forest_msu_test.py \
 random_forest_msu_test_math.py random_forest_msu_test_ref.py \
 --cov=random_forest_msu --cov-branch --cov-report=term-missing
```

2.2. Architecture, inheritance, and dependencies

`RandomForestMSU` extends `RandomForest` and overrides the construction and fitting of its base estimators, which are instances of `TreeClassifierMSU`. The latter extends `TreeClassifier` from `random_tree.py`, while `TreeMSU` extends `Tree` from that same corrected module. The generic `RandomForest` module also defines separate, non-VI versions of `Tree` and `TreeClassifier`. This is architectural duplication, but the corrected RFMSU VI instance builds its MSU trees from the corrected tree branch.

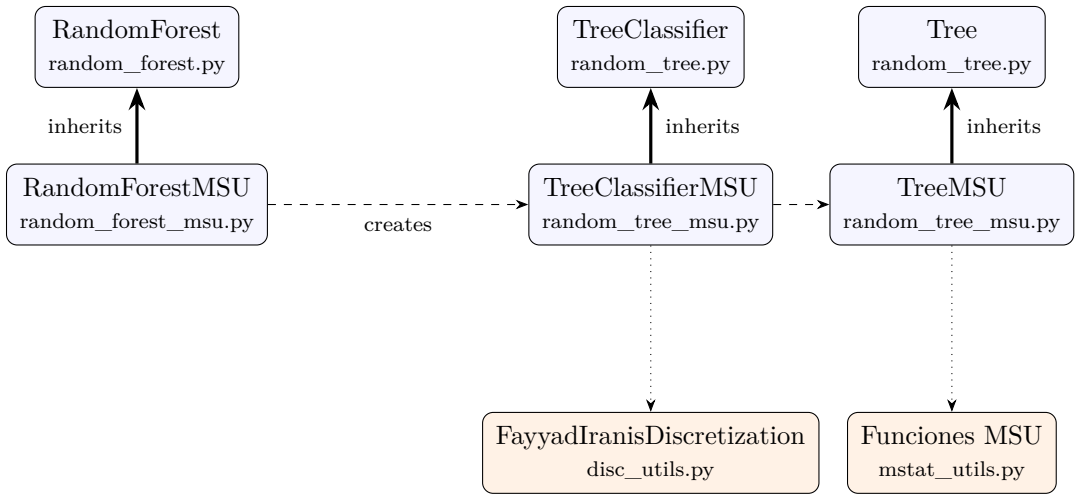


Figure 2.1: Class inheritance (solid lines) and usage relationships (dashed/dotted lines). An inheritance arrow points toward the base class. Schematic diagram.

The `stat_utils.py` module provides entropy functions; `mstat_utils.py` implements MSU; `disc_utils.py` provides MDL/Fayyad–Irani discretization. `joblib` enables optional parallel execution, and `numba` optionally accelerates selected discretization operations.

2.3. Classes and principal methods

The following inventory includes all eleven classes identified in the supplied implementation and its direct dependencies. Methods listed are defined on each class itself; inherited methods remain available through Python inheritance.

2.3.1. `RandomForestMSU` — `random_forest_msu.py`

Base class: `RandomForest`. Defined methods: `__post_init__, _make_tree, _fit_msu_tree_from_bootstrap_spec, fit, get_params, set_params, to_string, __str__`. Manages MSU-specific configuration, bootstrap modes, and the base-estimator factory.

2.3.2. `Tree` — `random_forest.py`

An independent tree implementation within the generic forest module; not the corrected VI tree. Methods: `weka_gr, weka_eq, is_missing_value, gain, prior_val, distribution, split_data, _normalized_distributio distribution_for_instance, build_tree, _leaf_string, _format_number, to_string, num_nodes, num_leaves, used_attributes, summary_string`.

2.3.3. `TreeClassifier` — `random_forest.py`

Independent generic-tree estimator, with `fit`, `predict`, `predict_proba`, `distribution_for_instance`, type inference, categorical metadata, representation, and tree-complexity methods.

2.3.4. `RandomForest` — `random_forest.py`

Defines `fit`, `predict`, `predict_proba`, `_compute_oob_score`, `_aligned_tree_proba`, bootstrap construction, serial/parallel training, estimator retrieval, impurity-decrease aggregation, tree-complexity metrics, parameter handling, and text representation. `RandomForestMSU` reuses these general forest behaviors.

2.3.5. `TreeMSU` — `random_tree_msu.py`

Base class: corrected VI `Tree`. Defines `weka_gr, weka_eq, _discretize_msu_for_node, distribution, _extend_msu_subset, _msu_score, split_data_msu, build_tree_msu`. It selects attributes using multivariate symmetrical uncertainty while keeping the auxiliary discretized matrix synchronized with the original observations.

2.3.6. `TreeClassifierMSU` — `random_tree_msu.py`

Base class: corrected VI `TreeClassifier`. Defines `_is_missing_value, _discretize_with_upo_disc, prepare_msu_discrete, fit`. It supports node-wise and forest-wide discretization workflows.

2.3.7. `Tree` — `random_tree.py`

The corrected tree node class implements entropy-based splitting, recursive tree growth, probability distributions, fallback predictions for unknown/empty categorical branches, textual output, and complexity counts. Principal methods include `distribution`, `split_data`, `distribution_for_instance`, `build_tree`, and `summary_string`.

2.3.8. `TreeClassifier` — `random_tree.py`

The corrected estimator wrapper provides `fit`, `predict`, `predict_proba`, `distribution_for_instance`, feature-type inference, category enumeration, textual output, and tree counts.

2.3.9. `MDLBasedDiscretizationModel` — `disc_utils.py`

Stores cut points and provides `__init__, get_cut_points, set_cut_points, __str__, discretize, transform`.

2.3.10. `MDLBasedDiscretization` — `disc_utils.py`

Abstract base class inheriting from `ABC`. Defines initialization, model construction, per-feature cut-point identification, `calculate_cut_points`, and `split_test`.

2.3.11. `FayyadIranisDiscretization` — `disc_utils.py`

Subclass of `MDLBasedDiscretization`. Implements the supervised MDL split-acceptance test through `split_test`.

2.4. Datasets

The three uploaded CSV files were inspected. The full dataset contains 77 observations (58 in class 1 and 19 in class 0). The training file contains 39 observations (29/10), and the test file 38 (29/9). All three have 2,648 columns: 2,647 numeric predictors plus the `class` target column. The supplied train/test split is preserved, and no test observations are used to fit the classifier. The demonstration ARFF files were generated from the first 20 predictors and the class column of the provided CSVs; they were *not* uploaded by the user.

Dataset	Rows	Predictors	Class counts (0/1)
Full	77	2647	19/58
Training	39	2647	10/29
Test	38	2647	9/29

2.5. Unit tests, mathematical verification, and scientific references

2.5.1. Unit tests — `random_forest_msu_test.py`

Unit tests verify inheritance, MSU estimator types, random seeds and repeatability, output shapes, `sample` and `weighted` bootstrap modes, bootstrap counts and indices, global discretization caching, explicitly supplied discretized matrices, parameter validation, out-of-bag (OOB) evaluation, sequential/parallel equivalence, and VI regression cases for unknown categories and empty branches.

2.5.2. Mathematical tests — `random_forest_msu_test_math.py`

Bootstrap counts satisfy $\sum_i c_i = B$, where B is the bag size. In standard probability averaging, the forest estimates

$$\widehat{P}(c \mid x) = \frac{1}{T} \sum_{t=1}^{T} \widehat{P}_t(c \mid x),$$

after aligning each estimator's class columns with the forest-wide class ordering. Majority voting instead uses

$$\widehat{P}_{\text{vote}}(c \mid x) = \frac{1}{T} \sum_{t=1}^{T} \mathbf{1}\{h_t(x) = c\}.$$

The OOB prediction for an observation includes only trees whose bootstrap sample excluded it. Tests reconstruct these quantities independently to verify the implementation.

2.5.3. Scientific-library comparisons — `random_forest_msu_test_ref.py`

Reference checks compare prediction shape, label handling, and performance on a controlled dataset against scikit-learn's `RandomForestClassifier`. That reference uses CART rather than MSU: these comparisons validate broad classifier behavior and performance, *not* identical learned splits. SciPy provides an independent check of the theoretical probability that a particular instance remains out-of-bag:

$$\Pr(C_i = 0) = \left(1 - \frac{1}{N}\right)^{B}.$$

```
python -m pytest -v random_forest_msu_test.py \
 random_forest_msu_test_math.py random_forest_msu_test_ref.py \
 --cov=random_forest_msu --cov-branch \
 --cov-report=term-missing --cov-report=html
```

2.6. Lymphoma example results

The following results were obtained from the included examples using 20 predictors, five trees, and the seeds specified in the corresponding scripts. Given the class imbalance, balanced accuracy should be considered alongside ordinary accuracy.

Experiment	Accuracy	Balanced accuracy
CSV train/test, global + sample	0.8158	0.6111
CSV train/test, local + sample	0.6842	0.4483
CSV train/test, global + weighted	0.7632	0.5383
ARFF train/test, global + sample	0.8158	0.6111
Stratified 5-fold CV (mean)	0.8200	0.7150

The illustrative OOB scores for global+sample, local+sample, and global+weighted were 0.8333, 0.5278, and 0.6944, respectively. These values depend on the feature subset and seeds and do not constitute a conclusive benchmark.

2.7. Runnable Python examples with syntax highlighting

Python keywords appear in blue, strings in orange, comments in green, and selected class names in purple. Each listing embeds the *exact executable Python file* shared with the Spanish chapter. The target column is named `class` throughout.

2.7.1. `comun.py`: shared helper functions

Loads CSV/ARFF files, selects predictor columns, configures the classifier, and reports evaluation metrics.

```python
"""Lectura reproducible de datos y construccion de RandomForestMSU VI.

Por defecto usa 20 atributos predictivos para demostraciones rapidas.
Para usar todos los atributos: --max-features 0.
"""
from pathlib import Path
import numpy as np
import pandas as pd
from scipy.io import arff
from rfmsu import RandomForestMSU

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / 'datos'

def select_features(frame, max_features=20):
    if 'class' not in frame.columns:
        raise ValueError('No se encontro la columna objetivo class')
    names = [c for c in frame if c != 'class']
    if max_features > 0:
        names = names[:max_features]
    return names

def load_csv(name, features=None, max_features=20):
    df = pd.read_csv(DATA / name)
    if features is None:
        features = select_features(df, max_features)
    X = df.loc[:, features].to_numpy(dtype=float)
    y = df['class'].to_numpy()
    return X, y, features

def load_arff(name, features=None, max_features=20):
    records, _ = arff.loadarff(DATA / name)
    df = pd.DataFrame(records)
    for col in df.columns:
        if df[col].dtype == object:
            df[col] = df[col].map(lambda v: v.decode('utf8') if isinstance(v, bytes) else v)
    if features is None:
        features = select_features(df, max_features)
    X = df.loc[:, features].to_numpy(dtype=float)
    y = df['class'].to_numpy()
    return X, y, features

def model(*, trees=5, seed=23, bootstrap='sample', local=False, oob=False,
          n_jobs=1, majority=False):
    return RandomForestMSU(
        debug=False, numIterations=trees, seed=seed, kValue=3,
        bootstrap_mode=bootstrap, rediscretize_msu_per_node=local,
        calcOutOfBag=oob, n_jobs=n_jobs, majority_vote=majority,
    )

def report(y_true, y_pred):
    from sklearn.metrics import accuracy_score, balanced_accuracy_score, confusion_matrix
    print('Accuracy:', round(accuracy_score(y_true, y_pred), 4))
    print('Balanced accuracy:', round(balanced_accuracy_score(y_true, y_pred), 4))
    print('Confusion matrix:\n', confusion_matrix(y_true, y_pred))
```

Listing 2.1: Shared utilities: comun.py

2.7.2. `01_csv_train_test.py`: separate training and test CSV files

Fits the classifier using the training CSV, evaluates only against the test CSV, and checks prediction probabilities and estimator count.

```python
"""Entrenamiento CSV y evaluacion en un CSV de test independiente."""
import argparse
from comun import load_csv, model, report

```

```
parser = argparse.ArgumentParser()
parser.add_argument('--max-features', type=int, default=20)
parser.add_argument('--trees', type=int, default=5)
args = parser.parse_args()

X_train, y_train, features = load_csv(
    '04-lymphoma_cancer_shipp_train.csv', max_features=args.max_features)
X_test, y_test, _ = load_csv(
    '04-lymphoma_cancer_shipp_test.csv', features=features)
clf = model(trees=args.trees, bootstrap='sample', local=False)
clf.fit(X_train, y_train, feature_types=['numeric'] * X_train.shape[1],
        feature_names=features)
pred = clf.predict(X_test)
report(y_test, pred)
print('Probability shape:', clf.predict_proba(X_test).shape)
print('Trees:', len(clf.estimators_))
```

Listing 2.2: CSV training and held-out testing

2.7.3. `02_csv_validacion_cruzada.py`: CSV cross-validation

Performs five-fold stratified cross-validation over the full CSV. Discretization is learned during each fold's training `fit` call rather than beforehand, avoiding supervised data leakage from validation observations.

```
"""Validacion cruzada estratificada sin discretizar fuera de cada fold."""
import argparse
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, balanced_accuracy_score
from comun import load_csv, model

parser = argparse.ArgumentParser()
parser.add_argument('--max-features', type=int, default=20)
parser.add_argument('--trees', type=int, default=5)
parser.add_argument('--folds', type=int, default=5)
args = parser.parse_args()
X, y, features = load_csv('04-lymphoma_cancer_shipp.csv',
                          max_features=args.max_features)
cv = StratifiedKFold(n_splits=args.folds, shuffle=True, random_state=2026)
acc, bacc = [], []
for fold, (train_idx, test_idx) in enumerate(cv.split(X, y), start=1):
    clf = model(trees=args.trees, seed=fold, local=False)
    clf.fit(X[train_idx], y[train_idx],
            feature_types=['numeric'] * len(features), feature_names=features)
    pred = clf.predict(X[test_idx])
    acc.append(accuracy_score(y[test_idx], pred))
    bacc.append(balanced_accuracy_score(y[test_idx], pred))
    print(f'Fold {fold}: accuracy={acc[-1]:.3f}, balanced={bacc[-1]:.3f}')
print(f'Accuracy media={np.mean(acc):.3f} +/- {np.std(acc):.3f}')
print(f'Balanced accuracy media={np.mean(bacc):.3f}')
```

Listing 2.3: Stratified cross-validation on CSV

2.7.4. `03_configuraciones.py`: different MSU and bootstrap configurations

Compares local versus global discretization and physical sampling versus weighted bootstrap; it also reports OOB measurements. The known limitation of unweighted MSU in weighted-bootstrap mode is retained.

```
"""Comparar discretizacion local/global, bootstrap sample/weighted y OOB."""
from comun import load_csv, model, report

X_train, y_train, names = load_csv('04-lymphoma_cancer_shipp_train.csv')
X_test, y_test, _ = load_csv('04-lymphoma_cancer_shipp_test.csv',
                             features=names)
for bootstrap, local in [('sample', False), ('sample', True),
                         ('weighted', False)]:
    clf = model(trees=5, seed=23, bootstrap=bootstrap, local=local,
```

```
                oob=True, n_jobs=1)
    clf.fit(X_train, y_train, feature_types=['numeric'] * len(names))
    print('\nBootstrap:', bootstrap, '| local discretization:', local)
    report(y_test, clf.predict(X_test))
    print('OOB score:', clf.oob_score_)
# Nota: en modo weighted, el MSU se calcula con frecuencias no ponderadas.
```

Listing 2.4: RFMSU configuration comparison

2.7.5. `04_arff_train_test.py`: separate training and test ARFF files

Uses SciPy to load demonstration ARFF files derived from the train/test CSV data, then fits and evaluates on corresponding feature columns.

```
"""Leer ARFF train/test con scipy.io.arff y evaluar el bosque."""
from comun import load_arff, model, report

X_train, y_train, features = load_arff('lymphoma_train_demo.arff')
X_test, y_test, _ = load_arff('lymphoma_test_demo.arff', features=features)
clf = model(trees=5, local=False)
clf.fit(X_train, y_train, feature_types=['numeric'] * len(features),
        feature_names=features)
report(y_test, clf.predict(X_test))
```

Listing 2.5: ARFF training and held-out testing

2.7.6. `05_arff_validacion_cruzada.py`: ARFF cross-validation

Loads the full ARFF file and performs stratified cross-validation with independent fitting in every fold.

```
"""Leer ARFF completo y ejecutar validacion cruzada estratificada."""
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score
from comun import load_arff, model

X, y, features = load_arff('lymphoma_full_demo.arff')
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=2026)
scores = []
for fold, (tr, te) in enumerate(cv.split(X, y), start=1):
    clf = model(trees=5, seed=fold, bootstrap='sample', local=False)
    clf.fit(X[tr], y[tr], feature_types=['numeric'] * len(features))
    score = accuracy_score(y[te], clf.predict(X[te]))
    scores.append(score)
    print(f'Fold {fold}: {score:.3f}')
print('Mean CV accuracy:', float(np.mean(scores)))
```

Listing 2.6: Stratified cross-validation on ARFF

2.7.7. `06_csv_todos_atributos.py`: all 2,647 predictor columns

Demonstrates full-feature training. This configuration may take considerably longer to run than the 20-feature examples.

```
"""Ejemplo de entrenamiento con todos los atributos (mas costoso)."""
from comun import load_csv, model, report
Xtr, ytr, names = load_csv('04-lymphoma_cancer_shipp_train.csv',
                           max_features=0)
Xte, yte, _ = load_csv('04-lymphoma_cancer_shipp_test.csv',
                       features=names)
clf = model(trees=10, local=False, n_jobs=1)
clf.fit(Xtr, ytr, feature_types=['numeric'] * len(names))
print('Number of predictors:', len(names))
report(yte, clf.predict(Xte))
```

Listing 2.7: CSV example using all predictors

2.8. Validity notes, recommendations, and limitations

1. In `bootstrap_mode=weighted`, the original MSU measure uses unweighted row frequencies rather than directly incorporating bootstrap multiplicities. For experiments where feature selection must reflect the resampling, `sample` is preferable.

2. Supervised discretization must not be fitted on the entire dataset before splitting into cross-validation folds, because that would leak information. The examples discretize within each training `fit`.

3. Further work should compare multiple tree counts, feature subsets, seeds, and datasets, and validate decisions against the Java/Weka implementation. Accuracy alone does not establish implementation equivalence.

4. The included ARFF files are illustrative reduced versions of the uploaded CSV data, not replacements for the original complete CSV files.